Support Vector Machine

PGR216

Mohib Ullah

19 March 2026

Content

Support Vector Machine

- Understand the intuition behind SVM
- Linear Classifier
- Maximum Margin
- Constrained optimization
- Quadratic programming
- Soft margin Vs hard margin
- SVM kernels
- SVM for Classification & Regression

Support Vector Machines

- SVMs are one of the popular fully supervised efficiently trained linear machines.

- They are famous because they are:
  - Effective in high dimensional spaces.
  - Effective in cases where number of dimensions is greater than the number of samples.
  - Very efficient in training stage. They use a subset of training points in the decision function and use the computer memory in an effective way.
  - Versatile: different kernel function can be specified for the decision function. Common kernels are provided, but it is also possible to specify custom kernels.

https://scikit-learn.org/stable/modules/svm.html

Support Vector Machines

- A supervised learning algorithm
- Used for:
  - Classification
  - Regression
- Finds an optimal decision boundary / function
- Maximizes margin between data points
- Uses kernel functions for non-linear problems

Linear Machine

- Equation of a linear function is written as:

$$Y = mx + c$$

$$f(x,w,b) = w^T x + b \text{ (linear machine)}$$

- Where m is slope of the line and c is the y intercept.

- However, the most generic equation of a linear machine/straight line is:

$$ax + by + c = 0$$

- If a = 0, the line is horizontal, and if b = 0, it would be a vertical line.

Decision Boundry

- For different configuration of parameter (a, b, c), there exist infinite solutions.

- Support Vector Machine (SVM) finds an optimal* solution.

  - Maximizes the distance between the (line, plane, or hyperplane, depending on dimensions) and the “difficult points” close to decision boundary

  - One intuition: if there are no points near the decision boundary, then we will have a very certain classification decisions

Two key attributes of SVMs are:

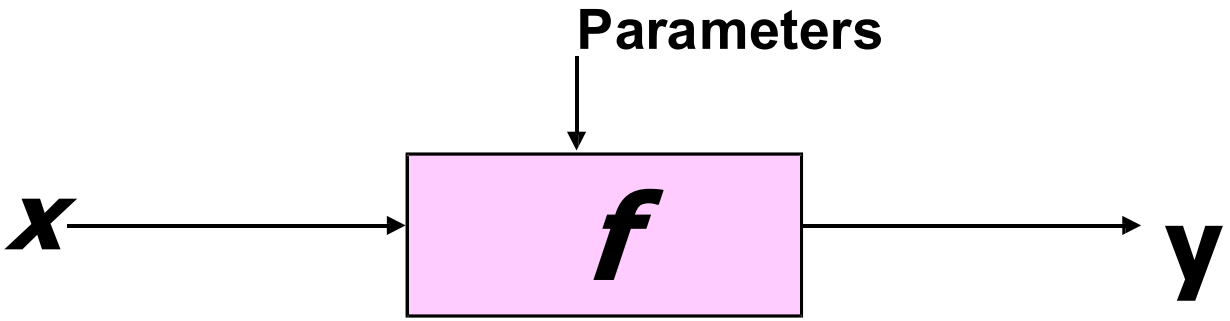
1. Sparseness

   Using subset of training points for training

2. Convexity

   Define a convex objective function.

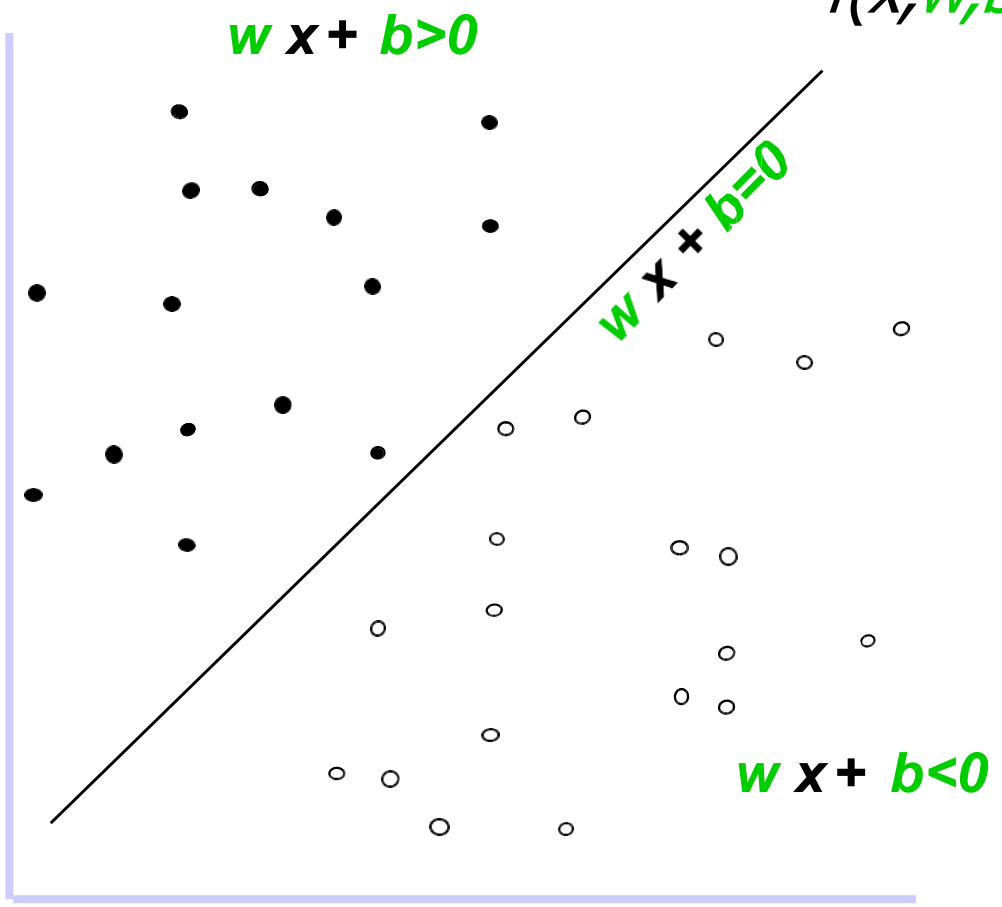
Linear Classifier



$f(x,w,b) = sign(w\,x + b)$

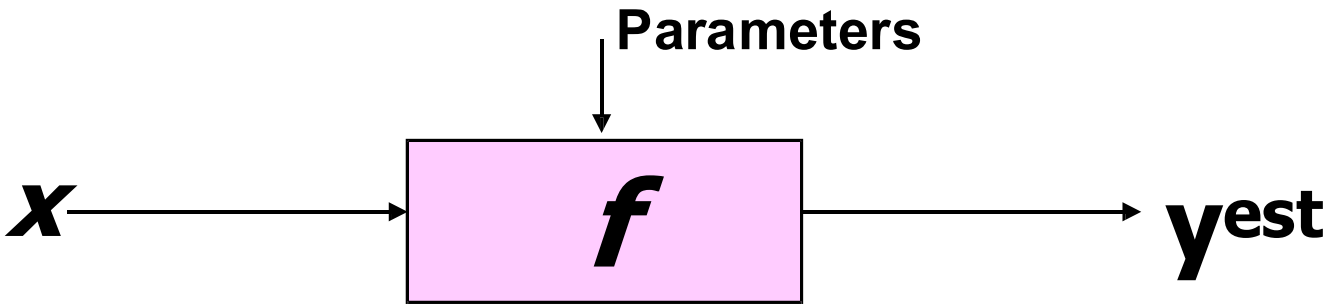
Label *y*:

- **denotes +1**
- **denotes -1**



How would you classify this data?

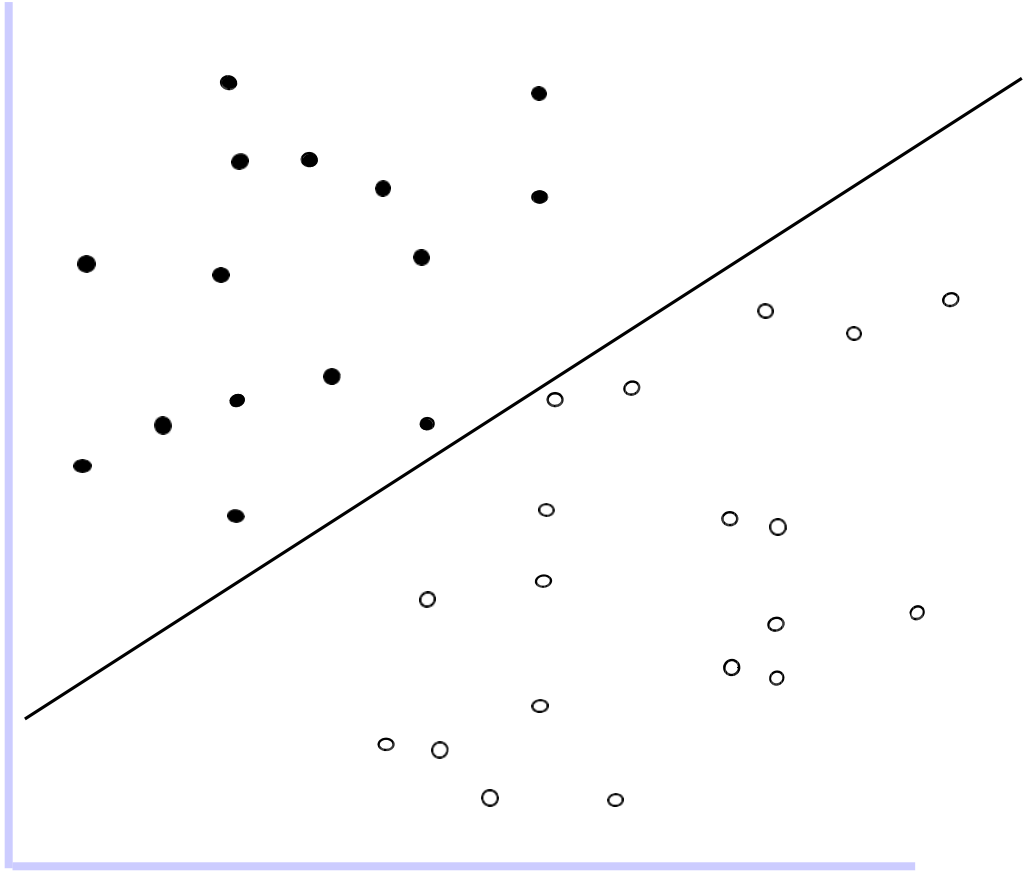
Linear Classifier



$f(x,w,b) = sign(w\,x + b)$

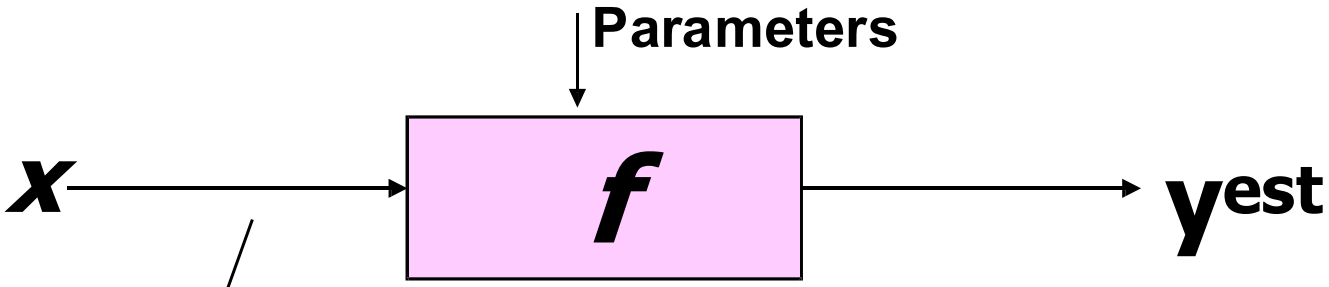
• **denotes +1**

◦ **denotes -1**

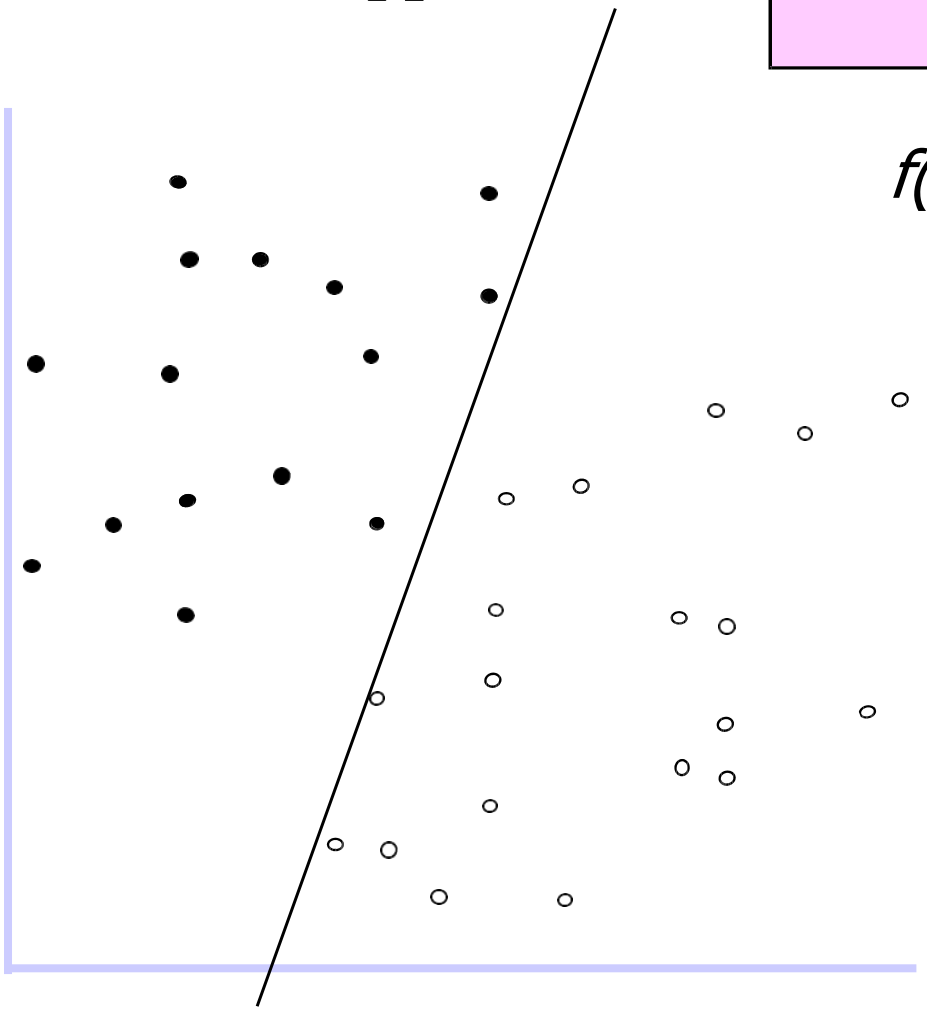


How would you classify this data?

Linear Classifier



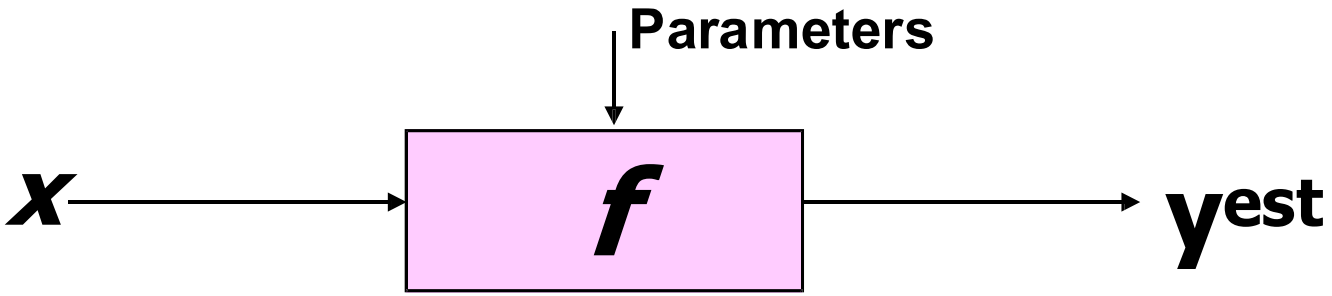
- • **denotes +1**
- ◦ **denotes -1**



$f(x,w,b) = sign(w\,x + b)$

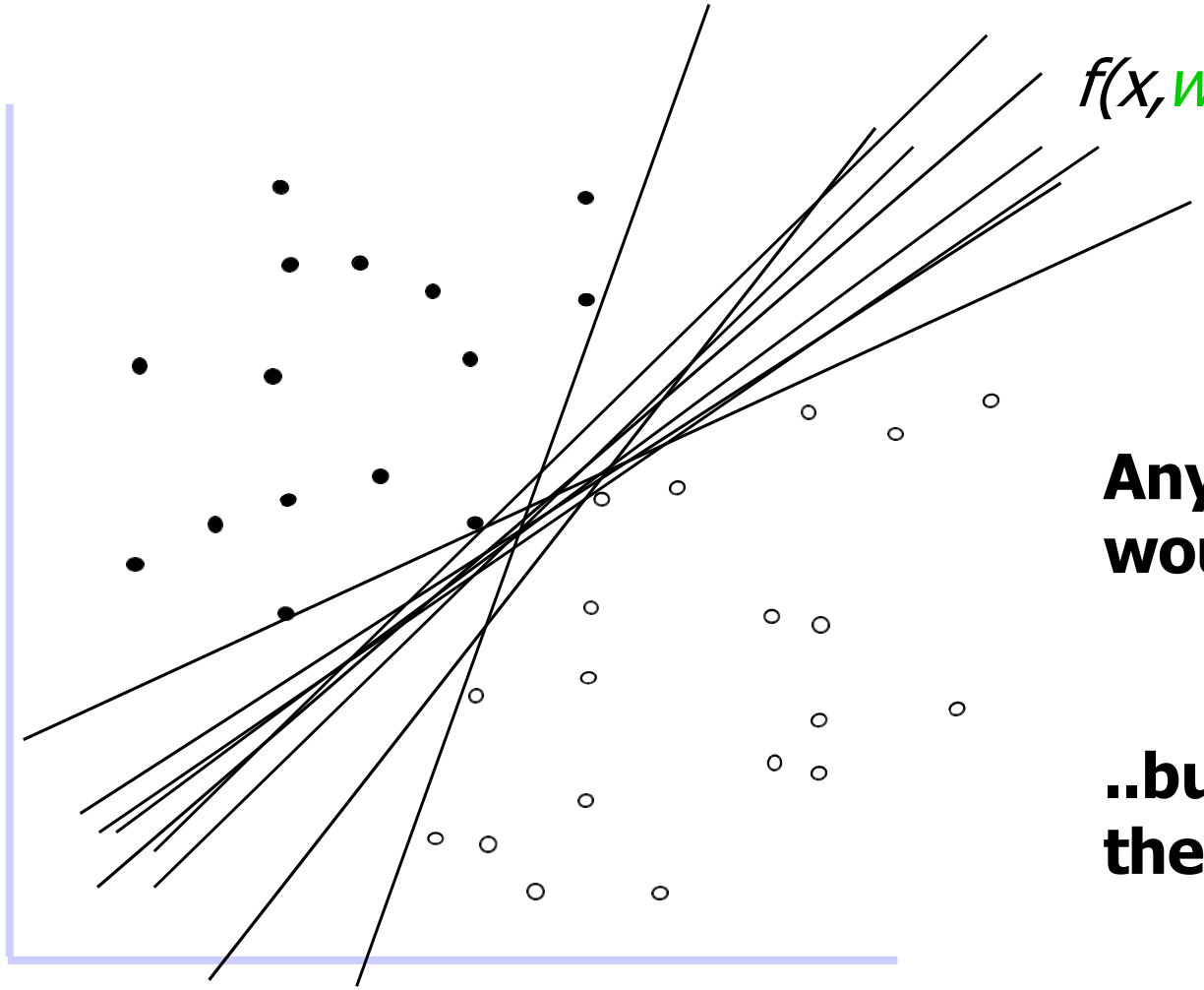
How would you classify this data?

Linear Classifiers



$f(x,w,b) = sign(w\,x + b)$

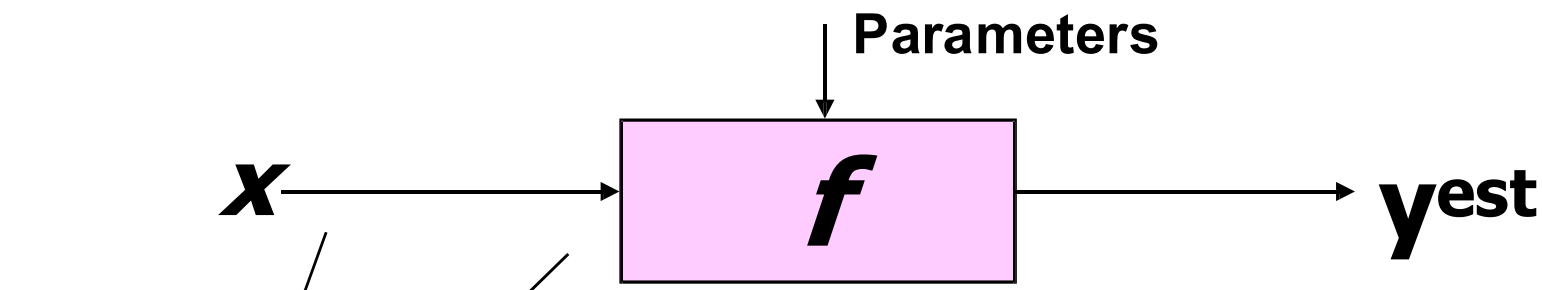
- • denotes +1
- ◦ denotes -1



Any of these would be fine..

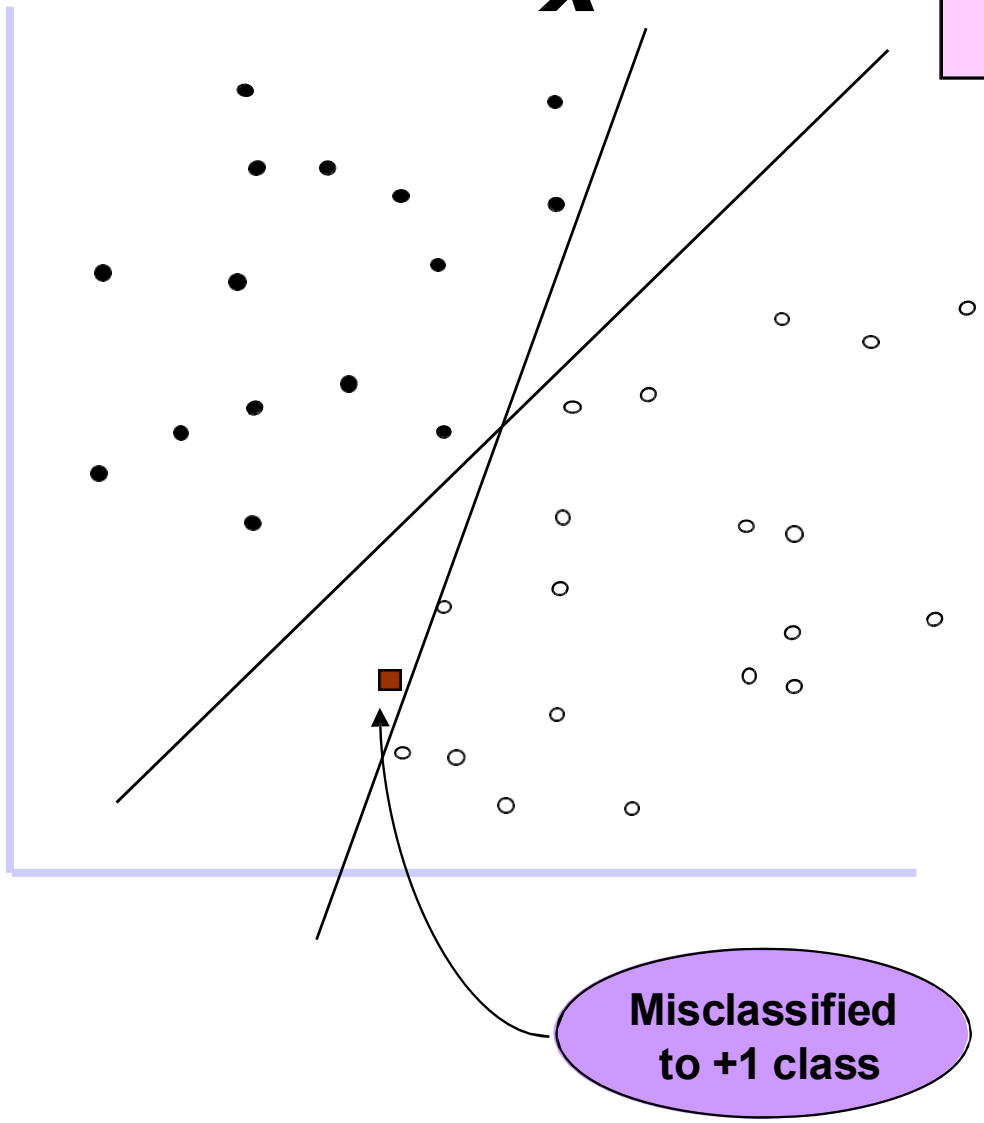
..but which is the best?

Linear Classifiers



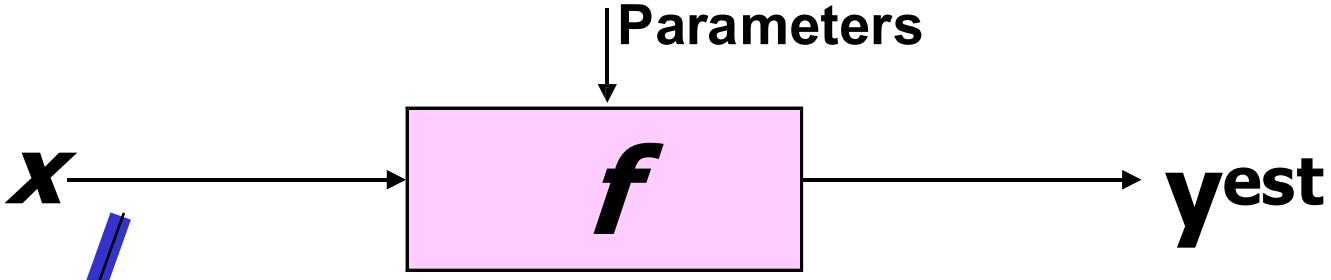
$f(x,w,b) = sign(w\,x + b)$

- • **denotes +1**
- ○ **denotes -1**



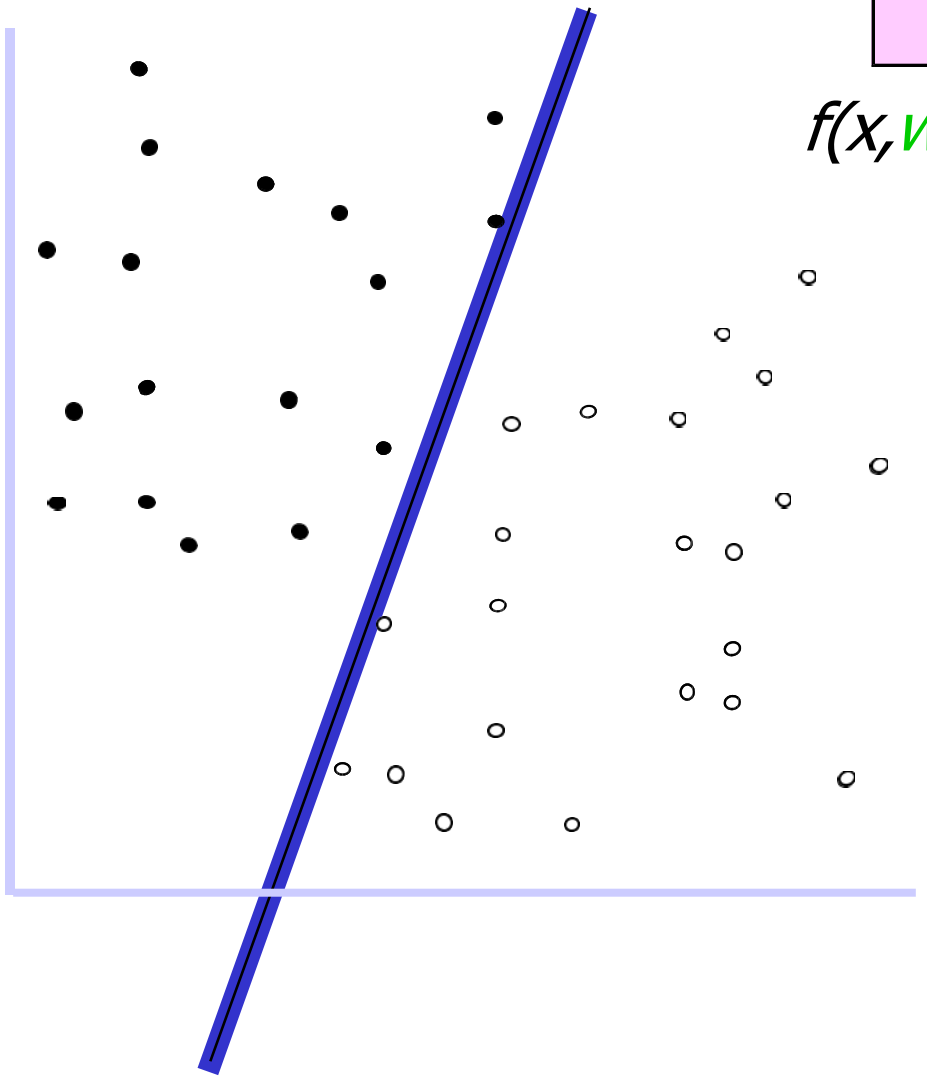
How would you classify this data?

Classifier Margin



$f(x,w,b) = sign(w\,x + b)$

- **denotes +1**
- **denotes -1**



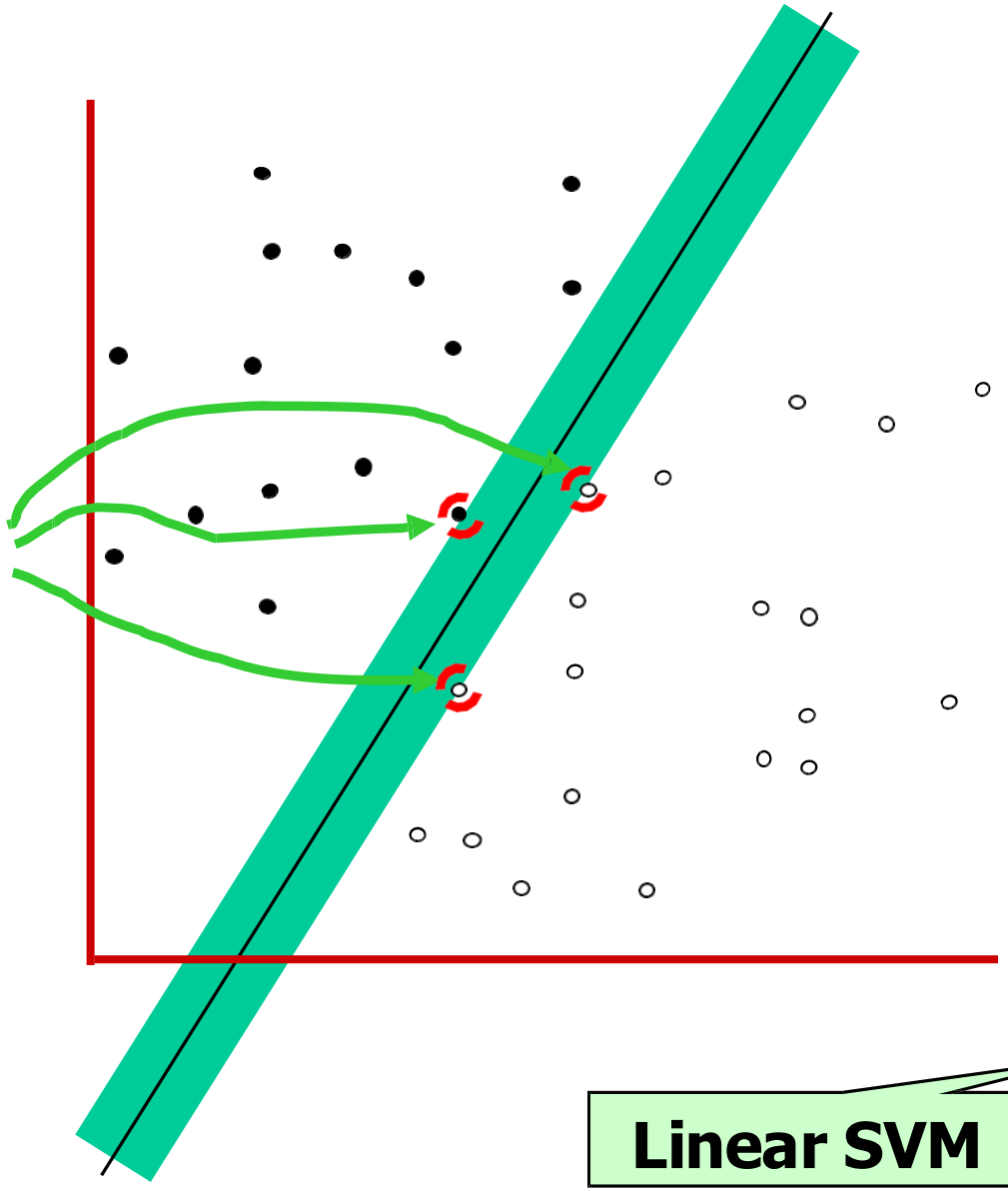
Define the margin of a linear classifier as the width that the boundary could be increased by before hitting a data point.

Maximum Margin

• **denotes +1**

◦ **denotes -1**

Support Vectors are those datapoints that the margin pushes up against



1. Maximizing the margin makes sense according to intuition
2. It implies that only support vectors are important; other training examples can be discarded without affecting the training result.

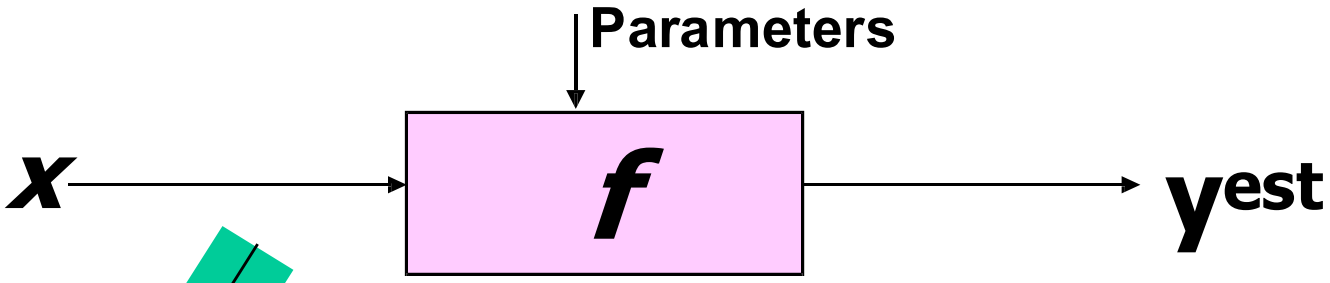
The maximum margin linear classifier is the linear classifier with the maximum margin.

This is the simplest kind of SVM (Called an LSVM)

Linear SVM

Maximum Margin

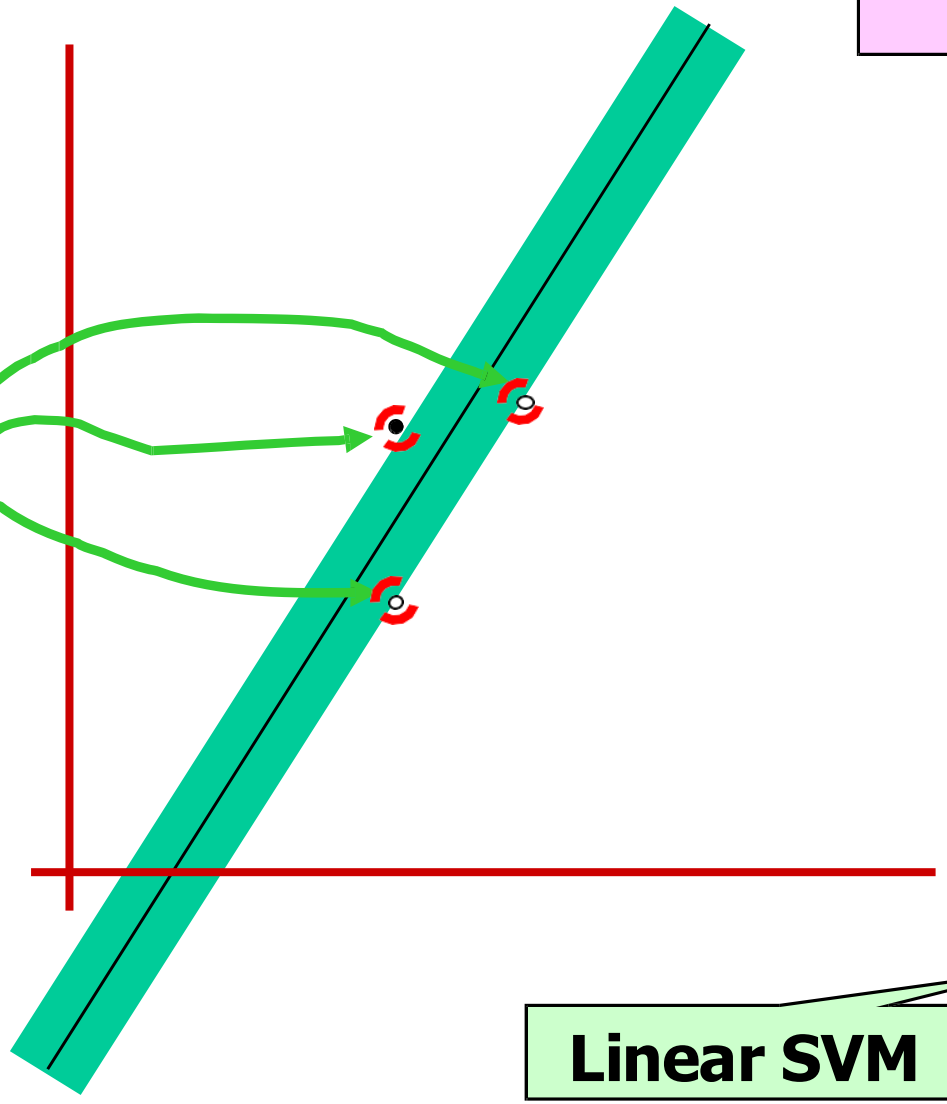
- • denotes +1
- ◦ denotes -1



$f(x,w,b) = sign(w\,x + b)$

Keeping only support vectors will not change the maximum margin classifier.

- ➢ Robust to the small changes (noise) in non-support vectors



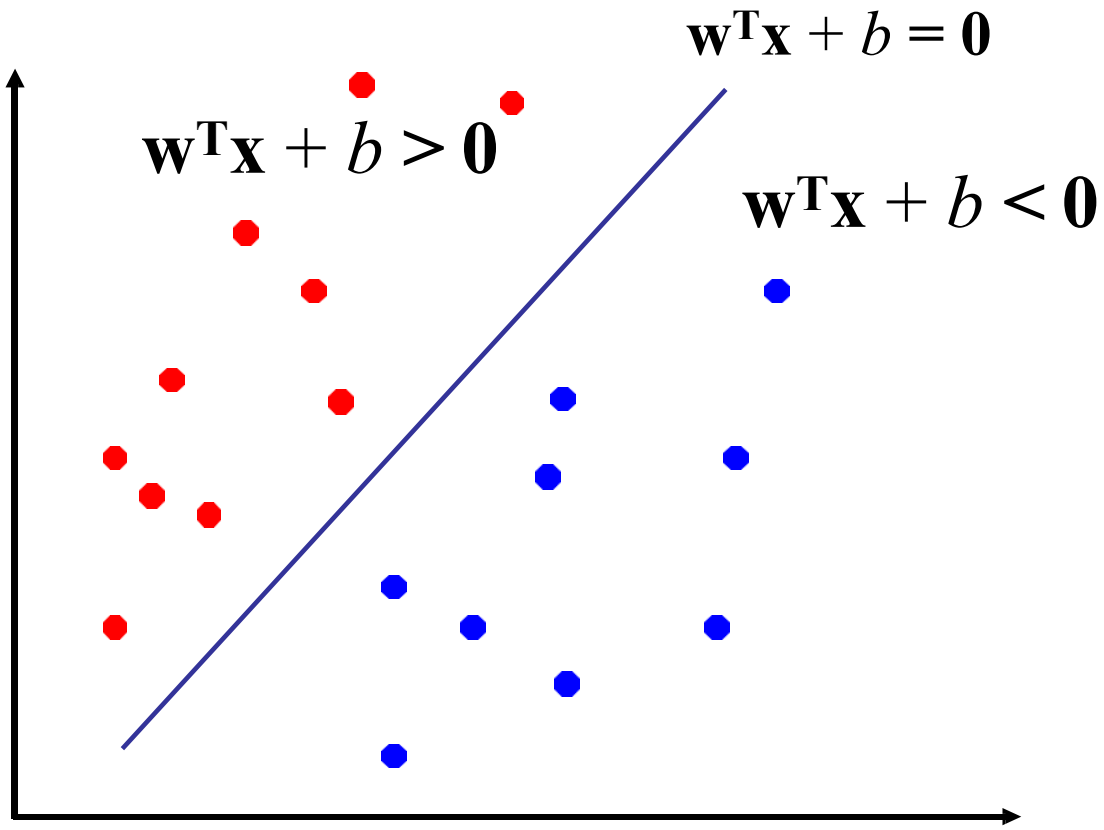
The maximum margin linear classifier is the linear classifier with the maximum margin.

This is the simplest kind of SVM (Called an LSVM)

Linear SVM

Linear Classifier

- Binary classifier → Task of separating classes in the feature space



$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x} + b)$$

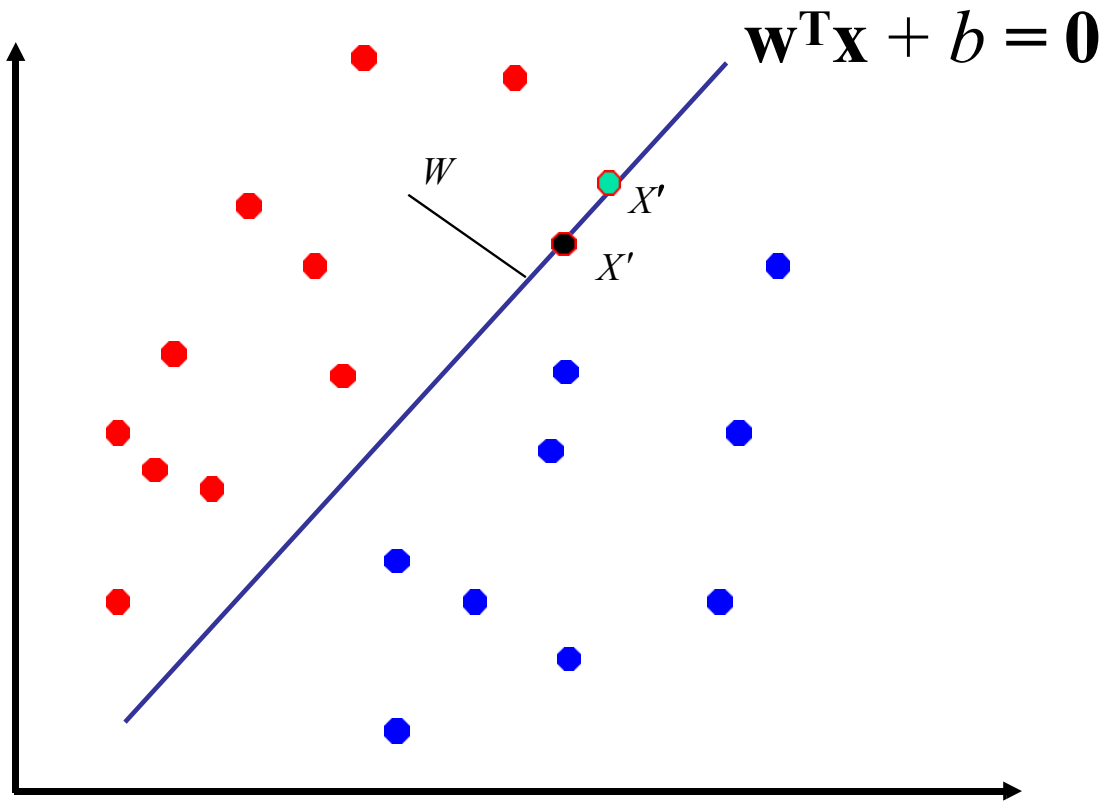
3 Important facts

- W is perpendicular to the line
- Distance from a point to a line is 1/|w|
- Distance between two lines (margin) is 2/|w|

We will proof this one by one.

W perpendicularity

- Finding perpendicular to the plane (line)



$$W^T X' + b = 0$$

$$W^T X' + b = 0$$

$$(W^T X' + b) - (W^T X' + b) = 0$$

$$W^T X' + b - W^T X' - b = 0$$

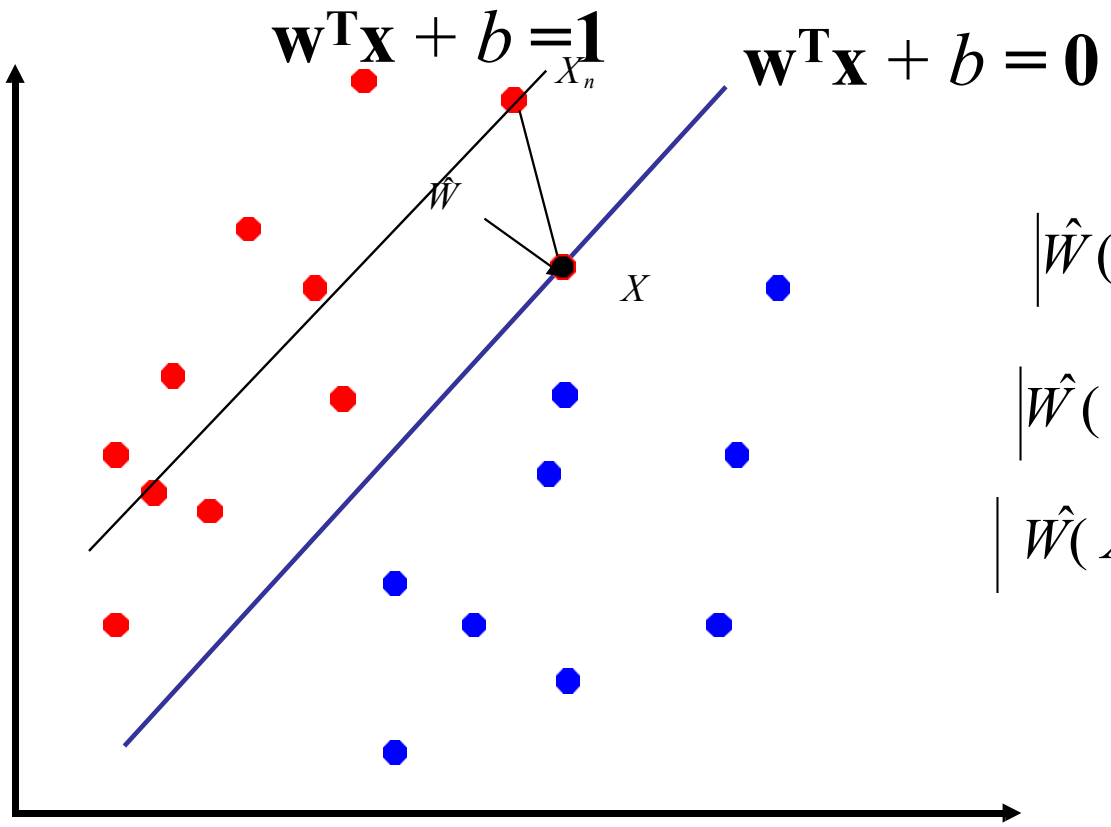
$$W^T X' - W^T X' = 0$$

$$W^T (X' - X') = 0$$

W is perpendicular a to line (hyper plane)

Distance from a point to a line is 1/|w|

- Binary classifier → Task of separating classes in feature space
- Distance from a point to a line (hyper plane)



$\hat{W} = \frac{W}{\|W\|}$ Unit vector

$$\left|\hat{W}(X - X_n)\right| = \left|\frac{w}{\|W\|}(X - X_n)\right|$$

$$\left|\hat{W}(X - X_n)\right| = \frac{1}{\|W\|}\left|(W.X - W.X_n)\right|$$

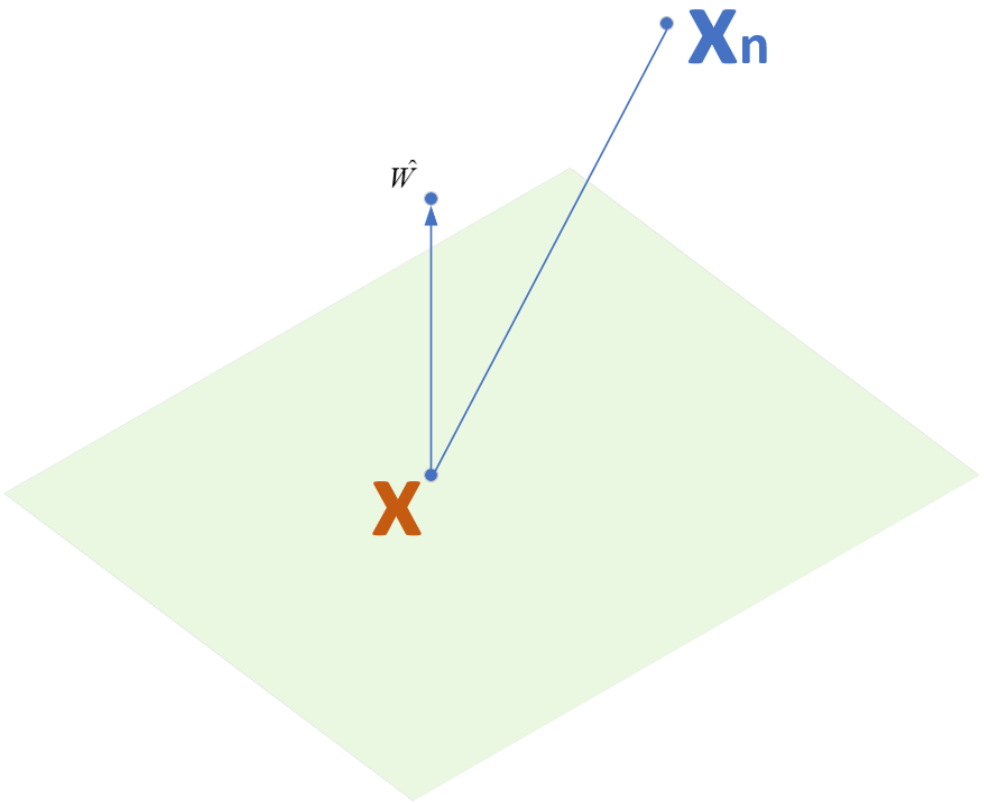
$$\left|\hat{W}(X - X_n)\right| = \frac{1}{\|W\|}\left|(W.X + b - W.X_n - b)\right|$$

$$\left|\hat{W}(X - X_n)\right| = \frac{1}{\|W\|}$$

Distance from a point to a line is 1/|w|

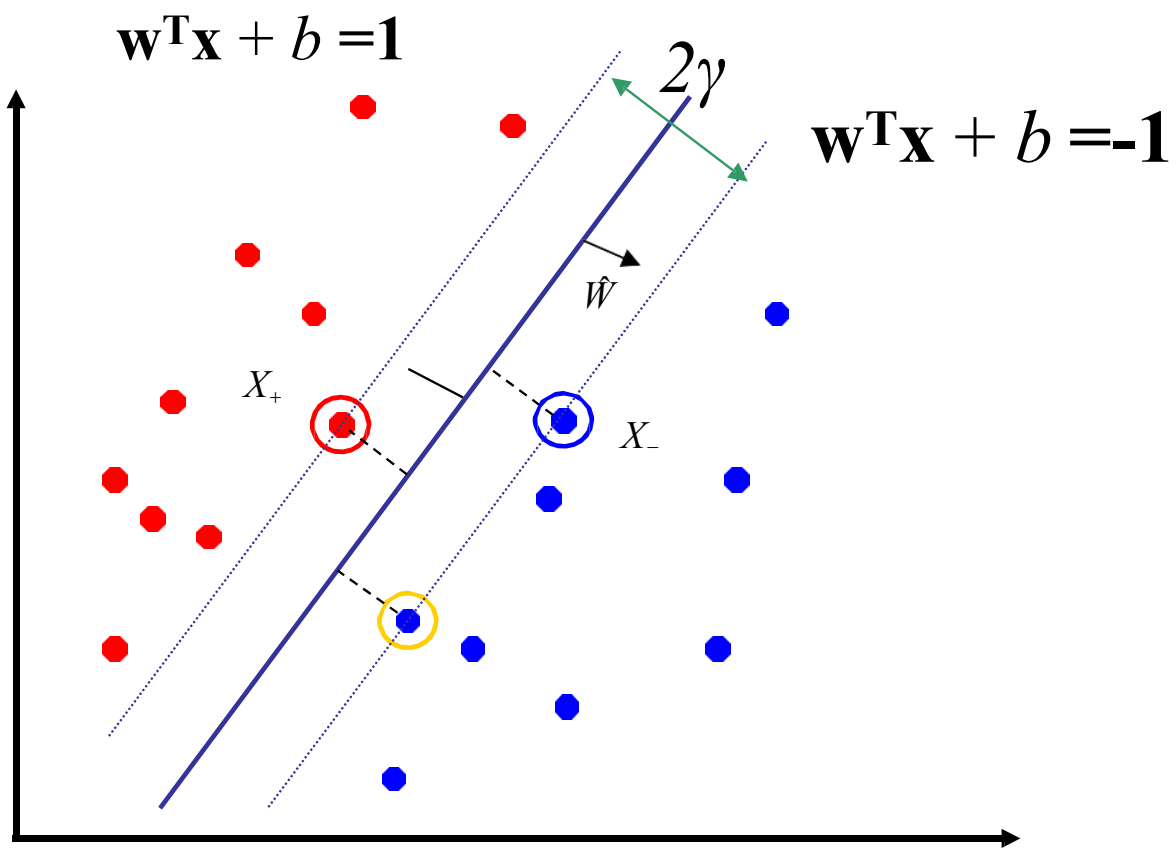
- Distance between x_n and the plane:
- Take any point x on the plane
- Projection of $(x_n - x)$ on w

$$\hat{W} = \frac{W}{||W||}$$



Distance between two lines (margin) is 2/|w|

- ***Margin*** 2γ of the separator is the width of separation between classes.



$$\hat{W}(X_+ - X_-) = \frac{W(X_+ - X_-)}{\|W\|}$$

$$\hat{W}(X_+ - X_-) = \frac{WX_+ - WX_-}{\|W\|}$$

$$\hat{W}(X_+ - X_-) = \frac{WX_+ + b - WX_- - b}{\|W\|}$$

$$\hat{W}(X_+ - X_-) = \frac{WX_+ + b - (WX_- + b)}{\|W\|}$$

$$\hat{W}(X_+ - X_-) = \frac{2}{\|W\|}$$

Linear SVM Mathematically

- **Goal:** **1) Correctly classify all training data**

$$wx_i + b \geq 1 \qquad \textit{if } y_i = +1$$

$$wx_i + b \leq -1 \qquad \textit{if } y_i = -1$$

$$y_i(wx_i + b) \geq 1 \qquad \text{for all i}$$

2) Maximize the Margin

$$M = \frac{2}{|w|}$$

same as minimize $\frac{1}{2} w^t w$

Linear SVM Mathematically

- We can formulate a Quadratic Optimization Problem and solve for w and b

Minimize $\Phi(w) = \frac{1}{2} w^t w$ **(Convexity)**

subject to $y_i(wx_i + b) \geq 1 \quad \forall i$

Quadratic programming

“Quadratic programming is the process of solving a special type of mathematical optimization problem specifically, a quadratic optimization problem, that is, the problem of optimizing a quadratic function of several variables subject to linear constraints on these variables.” (wiki)

Linear SVM

Now we can formulate the quadratic optimization problem as

Given a linearly separable training sample $S = ((\vec{\mathbf{x}}_1, y_1), \ldots (\vec{\mathbf{x}}_n, y_n))$,

the hyperplane $(\vec{\mathbf{w}}, b)$ that solves the optimization problem

$$\text{minimizes} \quad \langle \vec{\mathbf{w}} * \vec{\mathbf{w}} \rangle$$

$$\text{subject to} \quad y_i(\langle \vec{\mathbf{w}} * \vec{\mathbf{x}}_i \rangle + b) \geq 1 \qquad i = 1, \ldots n$$

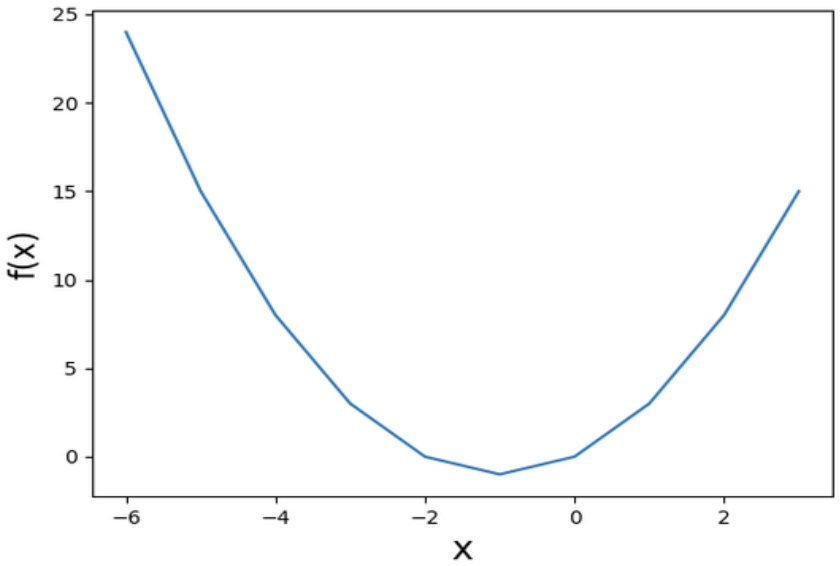
Detour (Unconstrained optimization)

- In order to find a maxima/minima of a function, we need to find the points where the gradient of the function is zero.

- If the function is convex, it will give us global maxima/minima.

- If the function is non-convex,

it will give us local maxima/minima.



Detour (Unconstrained optimization)

- If f(x) is a function which is constraints by another function g(x). Then, the gradient of the function f(x) and the gradient of the corresponding constraints g(x) are either parallel/anti-parallel vectors at the optimal point.

- In order to form the optimal value of function f(x), we need to write the function and it's constrained in a close form.

- Lagrange multiplier give us this close form representation.

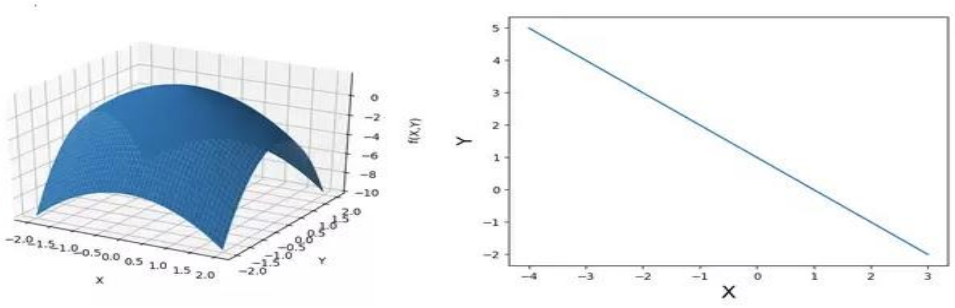
Detour: (Lagrange Multiplier)

- The method of Lagrange multipliers is a strategy for finding the local maxima/minima of a function subject to equality constraints.

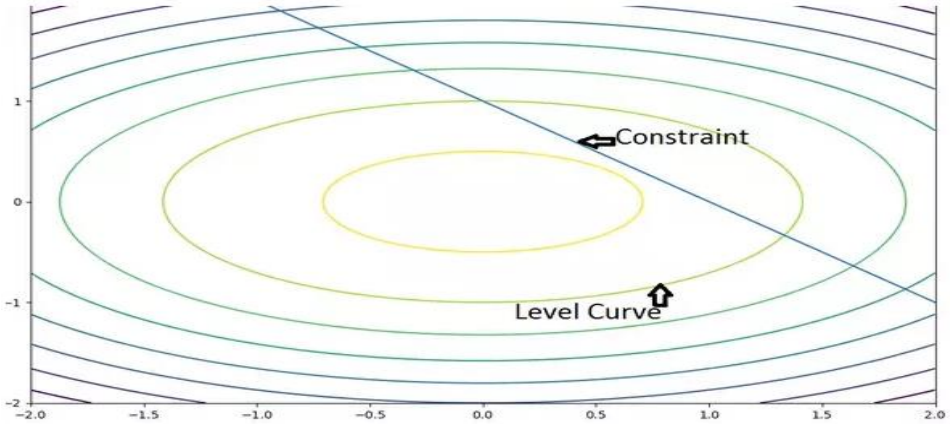
- Constraints are in the form of constants or another functions that have to be satisfied exactly by the chosen values of the variables to achieve the optimal value.

- For example:
  Optimize: $f(x, y) = 2 - x^2 - 2y^2$

- With the constraints: x + y = 1



The feasible set are the points which has to satisfy the constraint x + y = 1.



Linear SVM

minimizes $<\vec{\mathbf{w}} * \vec{\mathbf{w}}>$

subject to $y_i(<\vec{\mathbf{w}} * \vec{\mathbf{x}}_i> + b) \geq 1 \qquad i = 1,....n$

Convert the problem from the primal form into the dual form

$$L(\vec{\mathbf{w}},b,\vec{\boldsymbol{\alpha}}) = \frac{1}{2}<\vec{\mathbf{w}} * \vec{\mathbf{w}}> - \sum_{i=1}^{n} \alpha_i \left[y_i \left(<\vec{\mathbf{w}} * \vec{\mathbf{x}}_i> + b\right) - 1\right]$$

$$\frac{\partial L(\vec{w}^*,\vec{\alpha}^*,\vec{\beta}^*)}{\partial \vec{w}} = \vec{\mathbf{w}} - \sum_{i=1}^{n} y_i \alpha_i \vec{\mathbf{x}}_i = 0 \qquad \frac{\partial L(\vec{\mathbf{w}}^*,\boldsymbol{\alpha}^*,\vec{\boldsymbol{\beta}}^*)}{\partial b} = \sum_{i=1}^{n} y_i \alpha_i = 0$$

$$\vec{\mathbf{w}} = \sum_{i=1}^{n} y_i \alpha_i \mathbf{x}_i \qquad 0 = \sum_{i=1}^{n} y_i \alpha_i$$

Linear SVM

$$\vec{\mathbf{w}} = \sum_{i=1} y_i \alpha_i \mathbf{x}_i$$

- Now we plug the newly defined $\vec{\mathbf{w}}$ into the $L(\vec{\mathbf{w}}, b, \vec{\boldsymbol{\alpha}})$

$$L(\vec{\mathbf{w}}, b, \vec{\boldsymbol{\alpha}}) = \frac{1}{2}\langle \vec{\mathbf{w}} * \vec{\mathbf{w}} \rangle - \sum_{i=1}^{l} \alpha_i \left[y_i \left(\langle \vec{\mathbf{w}} * \vec{\mathbf{x}}_i \rangle + b \right) - 1 \right] =$$

$$\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} y_i y_j \alpha_i \alpha_j \langle \vec{\mathbf{x}}_i * \vec{\mathbf{x}}_j \rangle - \sum_{i=1}^{n} \sum_{j=1}^{n} y_i y_j \alpha_i \alpha_j \langle \vec{\mathbf{x}}_i * \vec{\mathbf{x}}_j \rangle + \sum_{j=1}^{n} \alpha_i =$$

$$\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} y_i y_j \alpha_i \alpha_j \langle \vec{\mathbf{x}}_i * \vec{\mathbf{x}}_j \rangle$$

Linear SVM

- The dual form of the original problem is

$$\text{maximize } W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} y_i y_j \alpha_i \alpha_j \langle \vec{\mathbf{x}}_i * \vec{\mathbf{x}}_j \rangle$$

$$\text{subject to} \quad \sum_{i=1}^{n} y_i \alpha_i = 0 \qquad \alpha_i \geq 0 \quad i = 1,\ldots,n.$$

The optimal weight vector given by: $\vec{\mathbf{w}}^* = \sum_{i=1}^{n} y_i \alpha^*_i \vec{\mathbf{x}}_i$

realizes the maximal margin hyperplane with the geometric margin given by: $\gamma = \dfrac{1}{\|\mathbf{w}\|_2}$

Solving the Optimization Problem

Find $\alpha_1 \ldots \alpha_N$ such that

$Q(\alpha) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

(1) $\Sigma\alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

Finding b from the linear constraints

$$y_k(W_T X_k + b) = 1$$

$$(W^T X_k + b) = \frac{1}{y_k}$$

$$(W^T X_k + b) = y_k$$

$$b = y_k - W^T X_k$$

The Optimization Problem Solution

- The solution has the form:

$$\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i \qquad b = y_k - \mathbf{w}^T \mathbf{x}_k \qquad \text{for any } \mathbf{x}_k \text{ such that } \alpha_k \neq 0$$

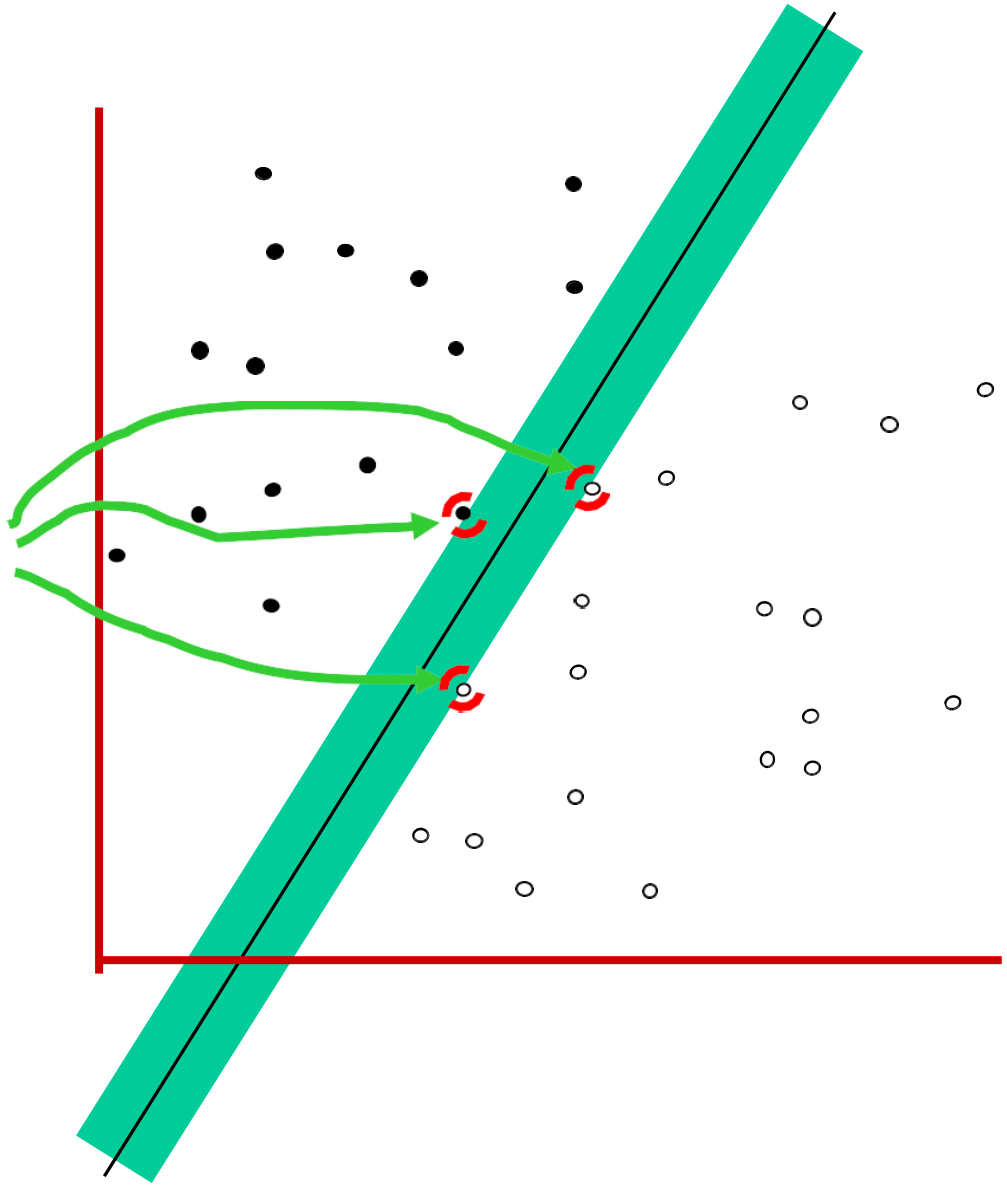
- α_i must satisfy Karush-Kuhn-Tucker conditions:
  $\alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0$, for any i
  - If $\alpha_i > 0$, $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0$, $\mathbf{x}_i$ is on the margin
  - If $y_i(\mathbf{w}^T x_i + b) > 1$, $\alpha_i = 0$ **(Sparsity)**
- Each non-zero α_i indicates that corresponding $\mathbf{x}_i$ is a support vector.

Maximum Margin

- **denotes +1**
- **denotes -1**

w, b depends only on Support Vectors via active constraints

$y_i(w^T x_k + b) - 1 = 0$



The Optimization Problem Solution

- To classify the new test point x, we use

$$f(x) = wx + b = \sum_{i \in SV}^{n} \alpha_i y_i \boxed{x_i x} + b$$

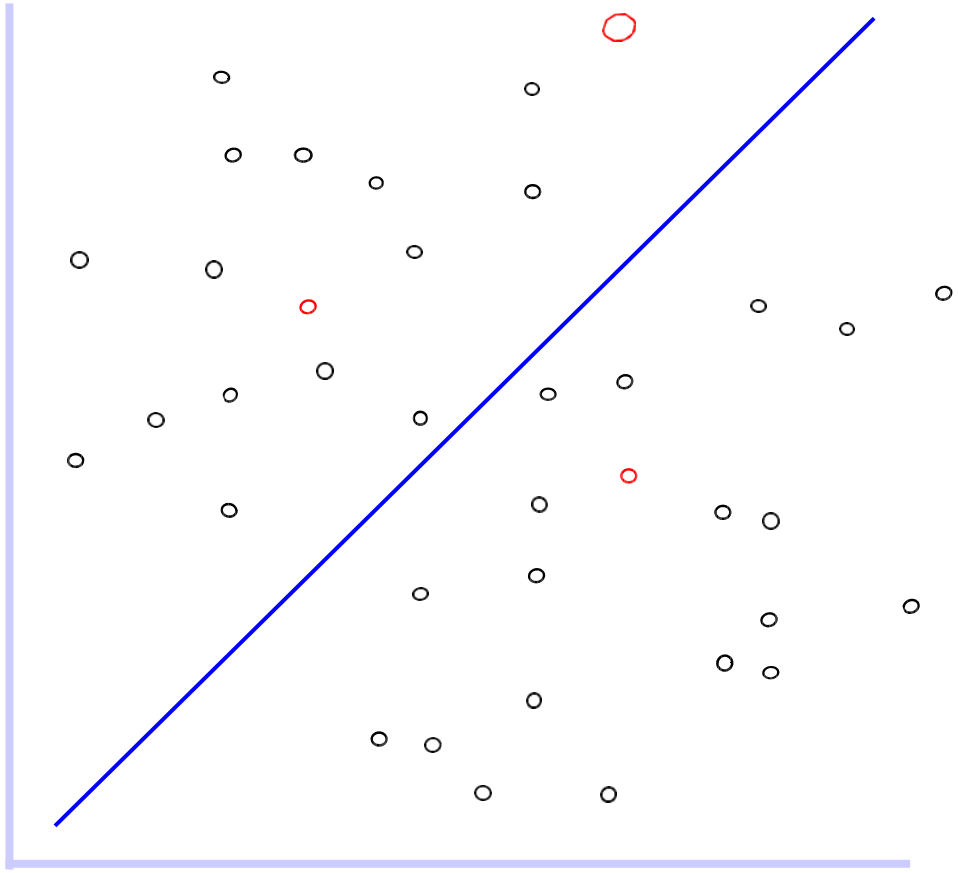
Find $\alpha_1 \ldots \alpha_N$ such that

$$Q(\alpha) = \Sigma\alpha_i - \tfrac{1}{2}\Sigma\Sigma\alpha_i \alpha_j y_i y_j \boxed{x_i^T x_j}$$ **is maximized and**

(1) $\Sigma\alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ **for all** α_i

Dataset with noise



- Hard Margin: So far , all data points are classified correctly

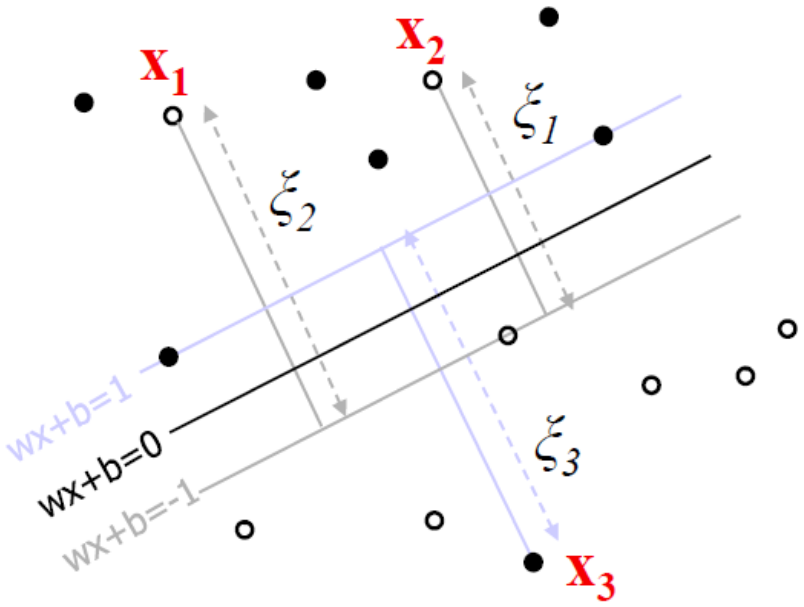
  - No training error

- What if the training set is noisy?

Soft Margin

- Introduce a slack variable in the soft margin.

$$y_i\,(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \qquad \xi_i \geq 0$$



- Slack variables quantify the miss-classification

What should our quadratic optimization criterion be?

We expect ξ_i to be small.

$$\Phi(\mathbf{w}) = \tfrac{1}{2}\,\mathbf{w}^T\mathbf{w} + C\Sigma\xi_i$$

Slide Credits: Guo-Jun QI

Hard Margin vs. Soft Margin

- The old formulation:

Find w and b such that

$\Phi(\mathrm{w}) = \frac{1}{2}\,\mathrm{w}^{\mathrm{T}}\mathrm{w}$ is minimized and for all $\{(\mathrm{x}_\mathrm{i}, y_i)\}$

$y_i(\mathrm{w}^{\mathrm{T}}\mathrm{x}_\mathrm{i} + \mathrm{b}) \geq 1$

- The new formulation incorporating slack variables:

Find w and b such that

$\Phi(\mathrm{w}) = \frac{1}{2}\,\mathrm{w}^{\mathrm{T}}\mathrm{w} + \boxed{C\Sigma\xi_i}$ is minimized and for all $\{(\mathrm{x}_\mathrm{i}, y_i)\}$

$y_i(\mathrm{w}^{\mathrm{T}}\mathrm{x}_\mathrm{i} + b) \geq 1 - \boxed{\xi_i}$ and $\xi_i \geq 0$ for all i

- Similar solution can be obtained to that of hard margin
- Parameter C can be viewed as a way to control overfitting.

Linear SVM:

- The classifier is essentially a separating hyperplane.
- The most "important" training points are the support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points $\mathbf{x_i}$ are support vectors with non-zero Lagrangian multipliers α_i.
- Both in the dual formulation of the problem and in the solution, training points appear only inside inner products:

Find $\alpha_1 \ldots \alpha_N$ such that

$\mathbf{Q}(\boldsymbol{\alpha}) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j \mathbf{x_i}^T\mathbf{x_j}$ is maximized and

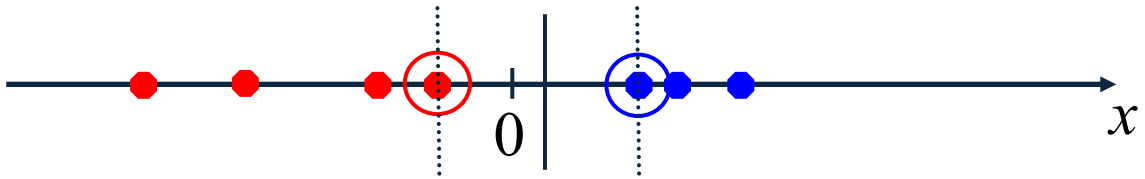
(1) $\Sigma\alpha_i y_i = 0$

(2) $0 \leq \alpha_i \leq C$ for all α_i

$$f(\mathbf{x}) = \Sigma\alpha_i y_i \mathbf{x_i}^T\mathbf{x} + b$$

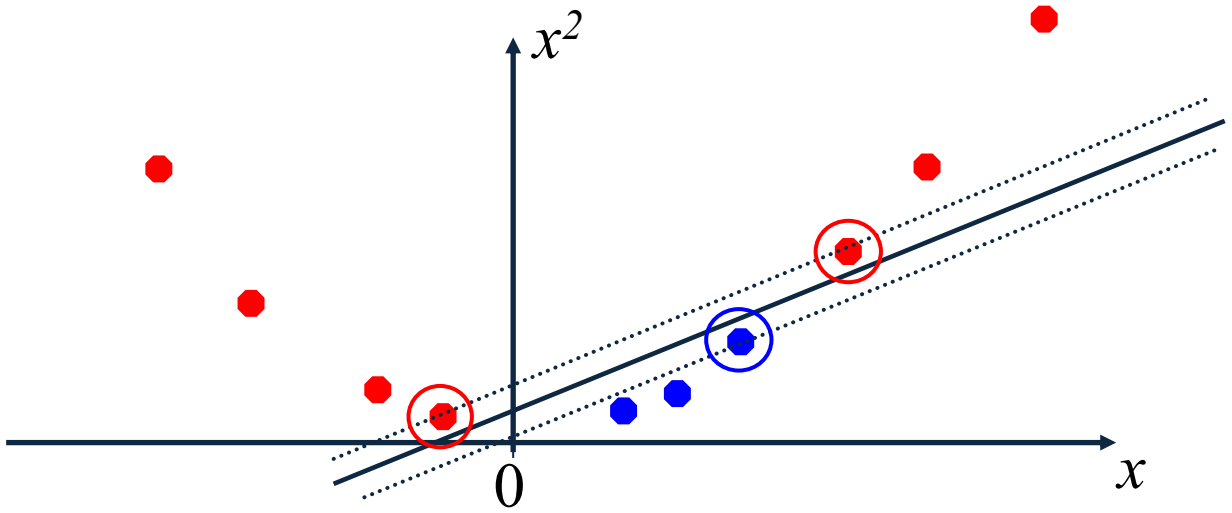
Non-linear SVM

Datasets that are linearly separable (with some noise) work out great:



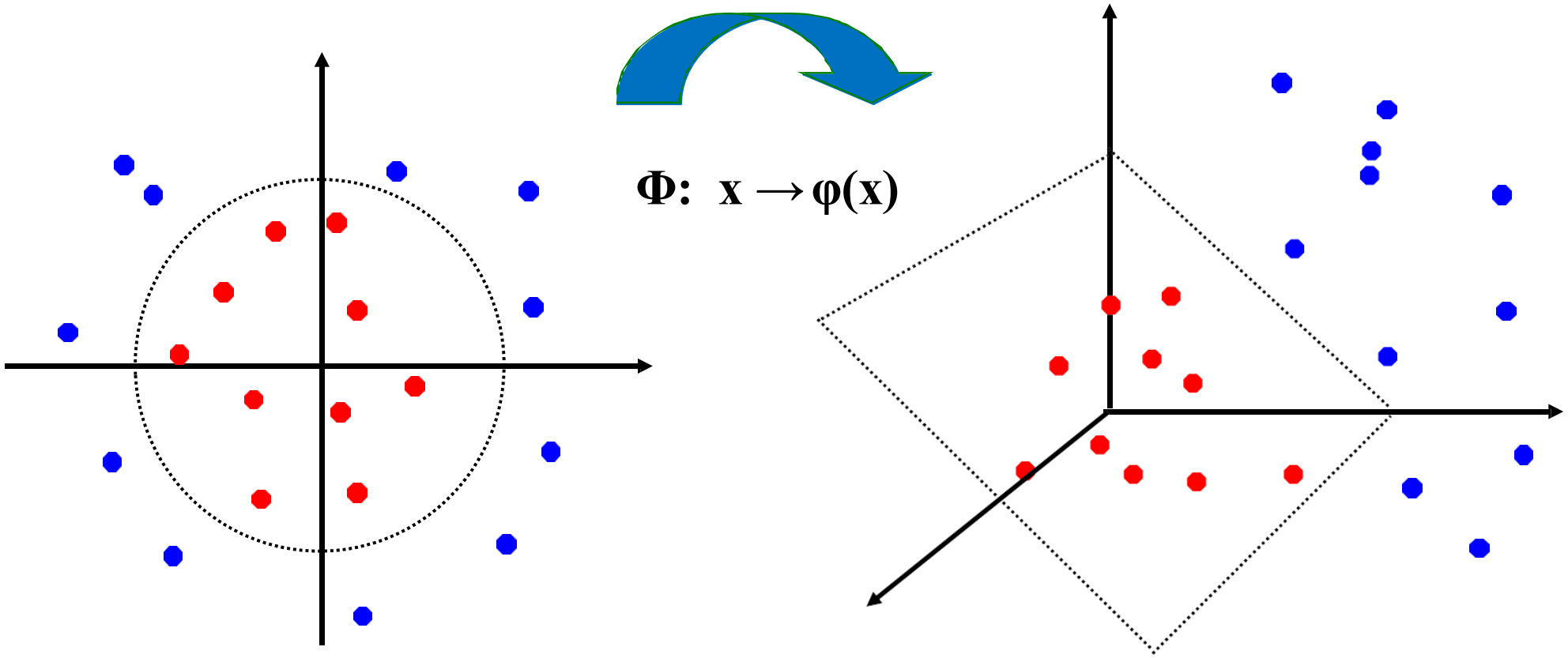
But what are we going to do if the dataset is just too hard?

How about … mapping data to a higher-dimensional space:



Non-linear SVMs: Feature spaces

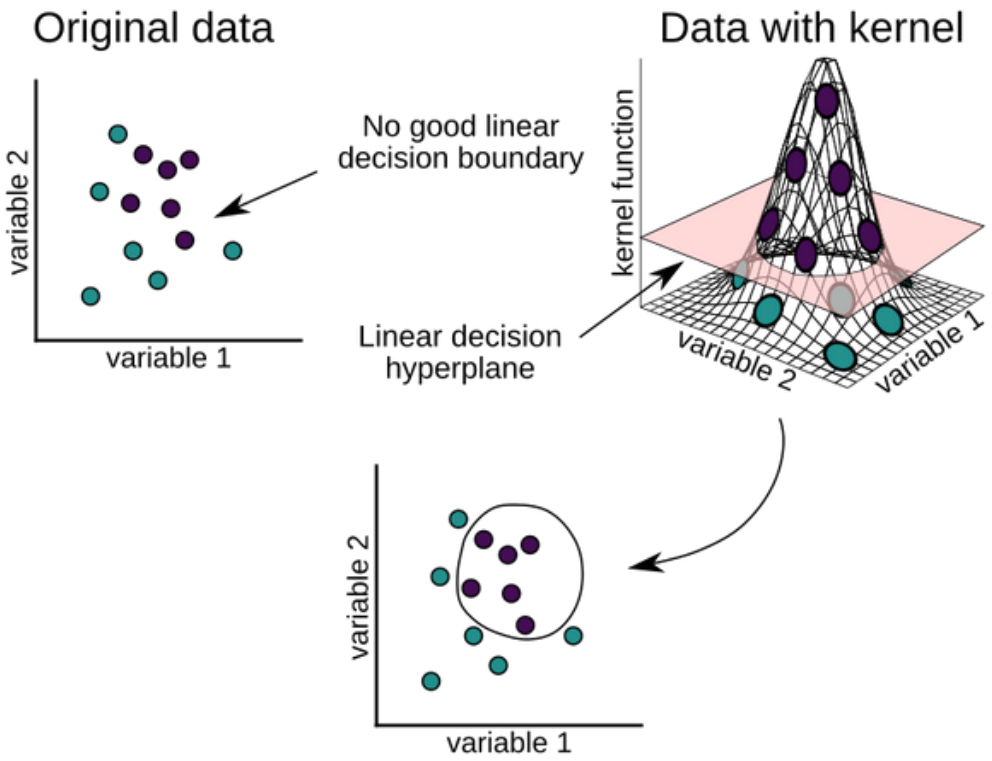
- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is linearly separable:



Kernels

- Why use kernels?
  - Make non-separable problem separable.
  - Map data into better representational space

- Common kernels
  - Linear
  - Polynomial $K(x,z) = (1+x^T z)^d$
  - Radial basis function (infinite dimensional space)
  - $K(x_i,x_j) = \exp-\{||x_i-x_j||^2\}/2\sigma^2$



https://www.r-bloggers.com/support-vector-machines-with-the-mlr-package/

Kernel Representation

In general, complex real-world applications require more expressive models than linear functions.

Kernel representation offers an alternative solution by projecting the data into a high (abstract) dimensional feature space to increase the computational power of the linear learning machines (i.e., classifiers and regressors).

The use of linear machines in the dual formulation makes it possible to apply the so-called kernel trick, i.e., to replace the inner product with an appropriate kernel function, performing thus implicitly the desired mapping into a feature space of higher dimensionality.

The “Kernel Trick”

- The linear classifier relies on an inner product between vectors $K(\mathbf{x}_i,\mathbf{x}_j)=\mathbf{x}_i^T\mathbf{x}_j$

- If every datapoint is mapped into high-dimensional space via some transformation

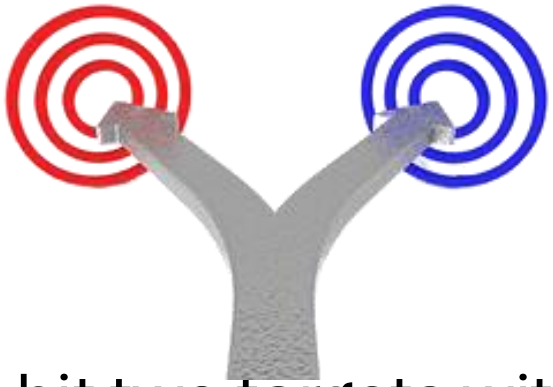
  Φ: $\mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i,\mathbf{x}_j)= \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$$

- A *kernel function* is some function that corresponds to an inner product in some expanded feature space.

The “Kernel Trick”

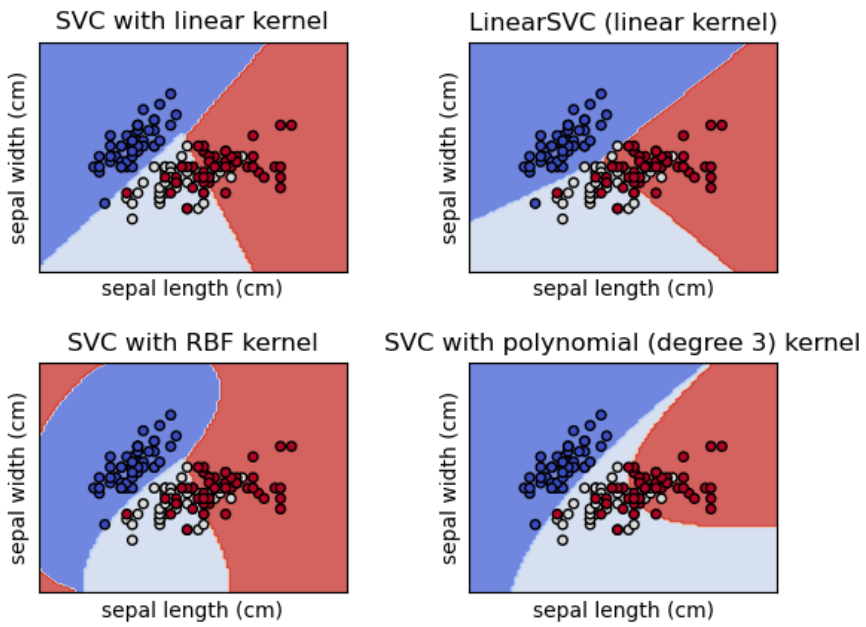
- In a nutshell, kernel trick allow us to operate in the original feature space without computing the coordinates of the date in a higher dimensional space.

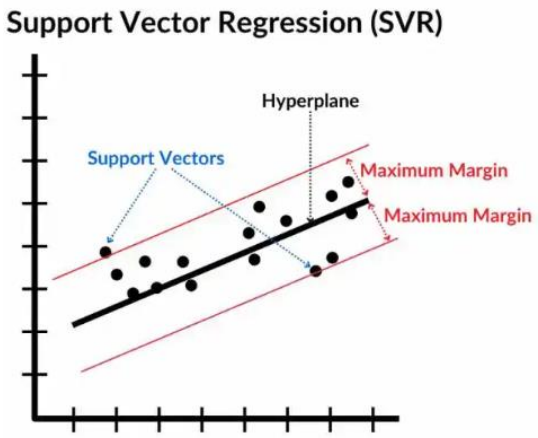


- So, with the kernel trick, we hit two targets with a singal shot. (data transformation, reduced computational)

SVM in classification and regression

- Support Vector Machines (SVMs) are used for:
  - **Classification → SVC (Support Vector Classifier)**
  - **Regression → SVR (Support Vector Regressor)**





https://scikit-learn.org/stable/modules/svm.html
https://spotintelligence.com/2024/05/08/support-vector-regression-svr/

SVM for Classification (SVC)

Objective

- Find a decision boundary with **maximum margin** between classes.
- **Optimization (soft margin):**

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum \xi_i$$

- subject to:

$$y_i(w^T\phi(x_i) + b) \geq 1 - \xi_i$$

Simple Python Example

```python
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# data
X, y = make_classification(n_samples=1000, n_features=5)

X_train, X_test, y_train, y_test = train_test_split(X, y)

# model
clf = SVC(kernel='rbf', C=1.0, gamma='scale')

# train
clf.fit(X_train, y_train)

# predict
y_pred = clf.predict(X_test)

# accuracy
print("Accuracy:", clf.score(X_test, y_test))
```

Important Parameters (Classification)

Parameter	Meaning
`C`	Regularization (higher → less margin, overfit)
`kernel`	`linear`, `rbf`, `poly`, `sigmoid`
`gamma`	Kernel width (rbf/poly)
`class_weight`	Handle class imbalance

SVM for Regression (SVR)

- SVR tries to fit a function such that:
  - ➢errors smaller than ε are ignored

- **Objective**

$$\min_{w,b,\xi,\xi^*} \frac{1}{2}||w||^2 + C\sum(\xi_i + \xi_i^*)$$

- subject to:

$$| y_i - f(x_i) | \leq \epsilon$$

Python Example (SVR)

```python
from sklearn.svm import SVR
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=1000, n_features=5, noise=10)

model = SVR(kernel='rbf', C=10, epsilon=0.1)

model.fit(X, y)

y_pred = model.predict(X[:5])
print(y_pred)
```

Important Parameters (Regression)

Parameter	Meaning
`C`	Error penalty
`epsilon`	Width of "no penalty" tube
`kernel`	Same as SVC
`gamma`	Kernel smoothness

Advantages of SVM

- Strong theoretical foundation
- Works well in high dimensions
- Effective with small datasets
- Robust to overfitting
- Flexible via kernels

Limitations

- Computationally expensive for large datasets
- Sensitive to hyperparameters
- Requires feature scaling
- Harder to interpret than linear models

When to Use SVM

- Use SVM when:
  - Dataset size is medium
  - Feature dimension is high
  - Clear margin exists
  - Accuracy is more important than speed

- Avoid when:
  - Dataset > 100k samples
  - Real-time training needed

Summary

- SVM finds maximum margin solutions
- Used for classification (SVC) and regression (SVR)
- Kernels enable non-linear learning
- Hyperparameters control bias–variance tradeoff
- Powerful but computationally demanding

Slides Adapted from:

- Pattern Recognition and Machine Learning Book by Christopher Bishop
- Kevin Murphy, UBC Machine learning course
- Mubarak Shah, UCF Computer Vision course
- Farid Melgani, UoT Pattern Recognition course
- Pattern Classification Book by Duda and Hart